

```

import os
import aiosqlite
from datetime import datetime
from aiogram import Bot, Dispatcher, types
from aiogram.types import Message, InlineKeyboardMarkup, InlineKeyboardButton
from aiogram.dispatcher import FSMContext
from aiogram.dispatcher.filters.state import State, StatesGroup
from aiogram.contrib.fsm_storage.memory import MemoryStorage
from aiogram.utils.executor import start_webhook

# — SOZLAMALAR —————
BOT_TOKEN = os.getenv("BOT_TOKEN")
PASSWORD = os.getenv("BOT_PASSWORD", "Sobirjon2005")
DB_NAME = "cloud.db"
ADMIN_ID = int(os.getenv("ADMIN_ID", "0"))

WEBHOOK_HOST = os.getenv("WEBHOOK_URL", "")
WEBHOOK_PATH = f"/webhook/{BOT_TOKEN}"
WEBHOOK_URL = f"{WEBHOOK_HOST}{WEBHOOK_PATH}"
WEBAPP_HOST = "0.0.0.0"
WEBAPP_PORT = int(os.getenv("PORT", 8000))

bot = Bot(token=BOT_TOKEN, parse_mode="HTML")
storage = MemoryStorage()
dp = Dispatcher(bot, storage=storage)

authorized_users = set()

# — HOLATLAR —————
class AuthState(StatesGroup):
    waiting_password = State()

class RenameState(StatesGroup):
    waiting_new_name = State()
    file_name = None

class ShareState(StatesGroup):
    waiting_user_id = State()
    file_name = None

# — DATABASE —————
async def create_db():
    async with aiosqlite.connect(DB_NAME) as db:
        await db.execute("""

```

```

        CREATE TABLE IF NOT EXISTS files (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER,
            file_id TEXT,
            file_name TEXT,
            category TEXT,
            size INTEGER,
            date TEXT,
            folder TEXT DEFAULT 'umumiy',
            pinned INTEGER DEFAULT 0,
            downloads INTEGER DEFAULT 0,
            description TEXT DEFAULT ''
        )
    """
    await db.execute("""
        CREATE TABLE IF NOT EXISTS folders (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT UNIQUE,
            date TEXT,
            icon TEXT DEFAULT '📁'
        )
    """)
    await db.execute("""
        CREATE TABLE IF NOT EXISTS trash (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER,
            file_id TEXT,
            file_name TEXT,
            category TEXT,
            size INTEGER,
            date TEXT,
            deleted_at TEXT
        )
    """)
    await db.commit()

async def save_file(user_id, file_id, file_name, category, size, folder="umumiy")
    async with aiosqlite.connect(DB_NAME) as db:
        await db.execute(
            "INSERT INTO files (user_id, file_id, file_name, category, size, date
            (user_id, file_id, file_name, category, size, datetime.now().strftime
            )
        )
        await db.commit()

# — YORDAMCHI —————
def is_auth(user_id):
    return user_id in authorized_users

```

```

def get_icon(cat):
    icons = {
        "video": "🎬", "photo": "🖼️", "apk": "📱",
        "ipa": "🍏", "audio": "🎵", "doc": "📄",
        "archive": "🗜️", "other": "📁"
    }
    return icons.get(cat, "📁")

def format_size(size):
    if size >= 1024 * 1024 * 1024:
        return f"{round(size / 1024 / 1024 / 1024, 2)} GB"
    elif size >= 1024 * 1024:
        return f"{round(size / 1024 / 1024, 2)} MB"
    elif size >= 1024:
        return f"{round(size / 1024, 1)} KB"
    return f"{size} B"

def get_category(ext):
    ext = ext.lower()
    if ext in ["mp4", "mov", "avi", "mkv", "webm"]:
        return "video", "🎬"
    elif ext in ["jpg", "jpeg", "png", "gif", "webp", "heic"]:
        return "photo", "🖼️"
    elif ext == "apk":
        return "apk", "📱"
    elif ext == "ipa":
        return "ipa", "🍏"
    elif ext in ["mp3", "wav", "ogg", "flac", "m4a"]:
        return "audio", "🎵"
    elif ext in ["pdf", "doc", "docx", "txt", "xlsx", "pptx"]:
        return "doc", "📄"
    elif ext in ["zip", "rar", "7z", "tar", "gz"]:
        return "archive", "🗜️"
    return "other", "📁"

async def send_file(chat_id, file_id, cat, caption, keyboard=None):
    try:
        if cat == "video":
            await bot.send_video(chat_id, file_id, caption=caption, reply_markup=
        elif cat == "photo":
            await bot.send_photo(chat_id, file_id, caption=caption, reply_markup=
        elif cat == "audio":
            await bot.send_audio(chat_id, file_id, caption=caption, reply_markup=
        else:
            await bot.send_document(chat_id, file_id, caption=caption, reply_mark
    except Exception as e:

```

```

        await bot.send_message(chat_id, f"{caption}\n\n⚠ Fayl yuborishda xato: {

def main_menu():
    kb = InlineKeyboardMarkup(row_width=2)
    kb.add(
        InlineKeyboardButton("📁 Barcha fayllar", callback_data="list"),
        InlineKeyboardButton("📊 Statistika", callback_data="stats"),
        InlineKeyboardButton("📁 Papkalar", callback_data="folders"),
        InlineKeyboardButton("📌 Muhim fayllar", callback_data="pinned"),
        InlineKeyboardButton("🎬 Videolar", callback_data="videos"),
        InlineKeyboardButton("📷 Rasmlar", callback_data="photos"),
        InlineKeyboardButton("🎵 Audio", callback_data="audios"),
        InlineKeyboardButton("📝 Hujjatlar", callback_data="docs"),
        InlineKeyboardButton("🗑 Savat", callback_data="trash"),
        InlineKeyboardButton("❓ Yordam", callback_data="help"),
    )
    return kb

# — AUTH —————
@dp.message_handler(commands=["start"])
async def start(message: Message):
    if is_auth(message.from_user.id):
        await message.answer(
            f"☁ <b>Shaxsiy Bulut Xotirangiz</b>\n\n"
            f"Salom, <b>{message.from_user.first_name}</b>! 🤝\n"
            f"📁 Fayl yuboring — avtomatik saqlanadi!\n\n"
            f"Quyidagi tugmalardan foydalaning:",
            reply_markup=main_menu()
        )
    else:
        await message.answer("🔑 <b>Parolni kiriting:</b>")
        await AuthState.waiting_password.set()

@dp.message_handler(state=AuthState.waiting_password)
async def check_password(message: Message, state: FSMContext):
    if message.text == PASSWORD:
        authorized_users.add(message.from_user.id)
        await state.finish()
        await message.answer(
            "✅ <b>Xush kelibsiz!</b>\n\nBulut xotiraga kirdingiz! ☁",
            reply_markup=main_menu()
        )
    if ADMIN_ID and message.from_user.id != ADMIN_ID:
        await bot.send_message(
            ADMIN_ID,
            f"👤 Yangi foydalanuvchi kirdi:\n"
            f"ID: <code>{message.from_user.id}</code>\n"

```

```

        f"Ism: {message.from_user.full_name}"
    )
else:
    await message.answer("❌ Noto'g'ri parol! Qayta urinib ko'ring:")

# — CALLBACK HANDLER —————
@dp.callback_query_handler()
async def handle_callbacks(call: types.CallbackQuery):
    if not is_auth(call.from_user.id):
        await call.answer("🔒 Avval kiring!", show_alert=True)
        return

    data = call.data
    await call.answer()

    if data == "list":
        await show_files(call.message)
    elif data == "stats":
        await send_stats(call.message)
    elif data == "folders":
        await send_folders(call.message)
    elif data == "pinned":
        await show_files(call.message, "AND pinned=1")
    elif data == "videos":
        await show_files(call.message, "AND category='video'")
    elif data == "photos":
        await show_files(call.message, "AND category='photo'")
    elif data == "audios":
        await show_files(call.message, "AND category='audio'")
    elif data == "docs":
        await show_files(call.message, "AND category='doc'")
    elif data == "trash":
        await show_trash(call.message)
    elif data == "help":
        await send_help(call.message)
    elif data == "menu":
        await call.message.answer("🏠 Bosh menyu:", reply_markup=main_menu())
    elif data.startswith("pin_"):
        name = data[4:]
        await toggle_pin(call.message, name, 1)
    elif data.startswith("unpin_"):
        name = data[6:]
        await toggle_pin(call.message, name, 0)
    elif data.startswith("del_"):
        name = data[4:]
        await move_to_trash(call.message, name)
    elif data.startswith("restore_"):

```

```

        name = data[8:]
        await restore_from_trash(call.message, name)

# ———— FAYL QABUL ————
@dp.message_handler(content_types=types.ContentType.VIDEO)
async def handle_video(message: Message):
    if not is_auth(message.from_user.id): return
    v = message.video
    name = v.file_name or f"video_{datetime.now().strftime('%Y%m%d_%H%M%S')}.mp4"
    await save_file(message.from_user.id, v.file_id, name, "video", v.file_size)
    await message.answer(
        f"🎬 <b>Saqlandi!</b>\n"
        f"📁 {name}\n"
        f"💾 {format_size(v.file_size)}"
    )

@dp.message_handler(content_types=types.ContentType.PHOTO)
async def handle_photo(message: Message):
    if not is_auth(message.from_user.id): return
    p = message.photo[-1]
    name = f"photo_{datetime.now().strftime('%Y%m%d_%H%M%S')}.jpg"
    await save_file(message.from_user.id, p.file_id, name, "photo", p.file_size)
    await message.answer(
        f"📷 <b>Saqlandi!</b>\n"
        f"📁 {name}\n"
        f"💾 {format_size(p.file_size)}"
    )

@dp.message_handler(content_types=types.ContentType.AUDIO)
async def handle_audio(message: Message):
    if not is_auth(message.from_user.id): return
    a = message.audio
    name = a.file_name or f"audio_{datetime.now().strftime('%Y%m%d_%H%M%S')}.mp3"
    await save_file(message.from_user.id, a.file_id, name, "audio", a.file_size)
    await message.answer(
        f"🎵 <b>Saqlandi!</b>\n"
        f"📁 {name}\n"
        f"💾 {format_size(a.file_size)}"
    )

@dp.message_handler(content_types=types.ContentType.VOICE)
async def handle_voice(message: Message):
    if not is_auth(message.from_user.id): return
    v = message.voice
    name = f"voice_{datetime.now().strftime('%Y%m%d_%H%M%S')}.ogg"
    await save_file(message.from_user.id, v.file_id, name, "audio", v.file_size)
    await message.answer(f"🎙️ <b>Ovozli xabar saqlandi!</b>\n💾 {format_size(v.fi

```

```
@dp.message_handler(content_types=types.ContentType.DOCUMENT)
async def handle_document(message: Message):
    if not is_auth(message.from_user.id): return
    d = message.document
    name = d.file_name or "nomsiz_fayl"
    ext = name.split(".")[1] if "." in name else ""
    category, icon = get_category(ext)
    await save_file(message.from_user.id, d.file_id, name, category, d.file_size)
    await message.answer(
        f"{icon} <b>Saqlandi!</b>\n"
        f"📄 {name}\n"
        f"📁 {format_size(d.file_size)}"
    )
```

```
# — RO'YXAT —————
async def show_files(message, where="", order="date DESC", params=()):
    if not is_auth(message.from_user.id):
        await message.answer("🔒 Avval /start orqali kiring!")
        return
    async with aisqlite.connect(DB_NAME) as db:
        q = f"SELECT file_id, file_name, category, size, date, folder, pinned FRO
        async with db.execute(q, params) as cursor:
            rows = await cursor.fetchall()
    if not rows:
        await message.answer("🚪 Hozircha fayl yo'q.", reply_markup=main_menu())
        return
    for file_id, name, cat, size, date, folder, pinned in rows:
        pin = "📌 " if pinned else ""
        caption = (
            f"{pin}{get_icon(cat)} <b>{name}</b>\n"
            f"📄 {format_size(size)} | 🕒 {date}\n"
            f"📁 {folder}"
        )
        kb = InlineKeyboardMarkup(row_width=2)
        if pinned:
            kb.add(InlineKeyboardButton("📌 Belgini olish", callback_data=f"unpin
        else:
            kb.add(InlineKeyboardButton("📌 Muhim", callback_data=f"pin_{name}"))
        kb.add(InlineKeyboardButton("🗑 O'chirish", callback_data=f"del_{name}"))
        await send_file(message.chat.id, file_id, cat, caption, kb)
```

```
# — QIDIRISH —————
@dp.message_handler(commands=["search"])
async def cmd_search(message: Message):
    if not is_auth(message.from_user.id): return
    args = message.text.split(maxsplit=1)
```

```

if len(args) < 2:
    await message.answer("Misol: /search fayl_nomi")
    return
keyword = f"%{args[1]}%"
async with aiosqlite.connect(DB_NAME) as db:
    async with db.execute(
        "SELECT file_id, file_name, category, size, date, folder, pinned FROM
        (keyword,)"
    ) as cursor:
        rows = await cursor.fetchall()
if not rows:
    await message.answer(f"🔍 '<b>{args[1]}</b>' topilmadi.")
    return
await message.answer(f"🔍 <b>{len(rows)} ta natija topildi:</b>")
for file_id, name, cat, size, date, folder, pinned in rows:
    caption = f"{get_icon(cat)} <b>{name}</b>\n📄 {format_size(size)} | 📁 {f
    await send_file(message.chat.id, file_id, cat, caption)

# — O'CHIRISH (SAVAT) —————
async def move_to_trash(message, name):
    async with aiosqlite.connect(DB_NAME) as db:
        async with db.execute(
            "SELECT file_id, category, size, date FROM files WHERE file_name=?",
        ) as cursor:
            row = await cursor.fetchone()
        if row:
            file_id, cat, size, date = row
            await db.execute(
                "INSERT INTO trash (user_id, file_id, file_name, category, size,
                (message.chat.id, file_id, name, cat, size, date, datetime.now().
                )
            await db.execute("DELETE FROM files WHERE file_name=?", (name,))
            await db.commit()
            await message.answer(f"🗑️ <b>{name}</b> savatga ko'chirildi!\n/trash
        else:
            await message.answer(f"❌ '{name}' topilmadi!")

@dp.message_handler(commands=["delete"])
async def cmd_delete(message: Message):
    if not is_auth(message.from_user.id): return
    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /delete fayl_nomi")
        return
    await move_to_trash(message, args[1])

async def show_trash(message):

```

```

async with aisqlite.connect(DB_NAME) as db:
    async with db.execute(
        "SELECT file_id, file_name, category, size, deleted_at FROM trash WHE
        (message.chat.id,)
    ) as cursor:
        rows = await cursor.fetchall()
if not rows:
    await message.answer("🗑 Savat bo'sh!")
    return
await message.answer(f"🗑 <b>Savat ({len(rows)} ta fayl):</b>")
for file_id, name, cat, size, deleted_at in rows:
    caption = f"{get_icon(cat)} <b>{name}</b>\n📄 {format_size(size)}\n🗑 O'c
    kb = InlineKeyboardMarkup()
    kb.add(InlineKeyboardButton("🔄 Tiklash", callback_data=f"restore_{name}")
    await send_file(message.chat.id, file_id, cat, caption, kb)

@dp.message_handler(commands=["trash"])
async def cmd_trash(message: Message):
    if not is_auth(message.from_user.id): return
    await show_trash(message)

async def restore_from_trash(message, name):
    async with aisqlite.connect(DB_NAME) as db:
        async with db.execute(
            "SELECT user_id, file_id, category, size, date FROM trash WHERE file_
        ) as cursor:
            row = await cursor.fetchone()
        if row:
            user_id, file_id, cat, size, date = row
            await db.execute(
                "INSERT INTO files (user_id, file_id, file_name, category, size,
                (user_id, file_id, name, cat, size, date)
            )
            await db.execute("DELETE FROM trash WHERE file_name=?", (name,))
            await db.commit()
            await message.answer(f"🔄 <b>{name}</b> tiklandi!")
        else:
            await message.answer("❌ Topilmadi!")

@dp.message_handler(commands=["emptytrash"])
async def cmd_empty_trash(message: Message):
    if not is_auth(message.from_user.id): return
    async with aisqlite.connect(DB_NAME) as db:
        await db.execute("DELETE FROM trash WHERE user_id=?", (message.from_user.
        await db.commit()
    await message.answer("🗑 Savat tozalandi!")

```

```

# — PAKKALAR —————
@dp.message_handler(commands=["newfolder"])
async def cmd_newfolder(message: Message):
    if not is_auth(message.from_user.id): return
    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /newfolder ish")
        return
    name = args[1]
    async with aioredislite.connect(DB_NAME) as db:
        try:
            await db.execute(
                "INSERT INTO folders (name, date) VALUES (?,?)",
                (name, datetime.now().strftime("%Y-%m-%d %H:%M"))
            )
            await db.commit()
            await message.answer(f"📁 '<b>{name}</b>' papkasi yaratildi!")
        except:
            await message.answer(f"❌ '<b>{name}</b>' papkasi allaqachon bor!")

async def send_folders(message):
    async with aioredislite.connect(DB_NAME) as db:
        async with db.execute("SELECT name FROM folders") as cursor:
            rows = await cursor.fetchall()
    text = "📁 <b>Papkalar:</b>\n\n📁 umumiy\n"
    for (name,) in rows:
        text += f"📁 {name}\n"
    text += "\n/folder nom — papka ichini ko'rish"
    await message.answer(text)

@dp.message_handler(commands=["folders"])
async def cmd_folders(message: Message):
    if not is_auth(message.from_user.id): return
    await send_folders(message)

@dp.message_handler(commands=["folder"])
async def cmd_folder(message: Message):
    if not is_auth(message.from_user.id): return
    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /folder ish")
        return
    await show_files(message, "AND folder=?", params=(args[1],))

@dp.message_handler(commands=["moveto"])
async def cmd_moveto(message: Message):
    if not is_auth(message.from_user.id): return

```

```

args = message.text.split(maxsplit=1)
if len(args) < 2 or "|" not in args[1]:
    await message.answer("Misol: /moveto fayl_nomi|papka_nomi")
    return
parts = args[1].split("|")
file_name, folder_name = parts[0].strip(), parts[1].strip()
async with aioredis.connect(DB_NAME) as db:
    await db.execute("UPDATE files SET folder=? WHERE file_name=?", (folder_n
    await db.commit()
await message.answer(f"✅ <b>{file_name}</b> → 📁 {folder_name}")

@dp.message_handler(commands=["delfolder"])
async def cmd_delfolder(message: Message):
    if not is_auth(message.from_user.id): return
    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /delfolder papka_nomi")
        return
    name = args[1]
    async with aioredis.connect(DB_NAME) as db:
        await db.execute("UPDATE files SET folder='umumiy' WHERE folder=?", (name
        await db.execute("DELETE FROM folders WHERE name=?", (name,))
        await db.commit()
    await message.answer(f"🗑️ '<b>{name}</b>' papkasi o'chirildi! Fayllar 'umumiy

# — MUHIM FAYLLAR —————
async def toggle_pin(message, name, value):
    async with aioredis.connect(DB_NAME) as db:
        await db.execute("UPDATE files SET pinned=? WHERE file_name=?", (value, n
        await db.commit()
    if value:
        await message.answer(f"📌 <b>{name}</b> muhim belgilandi!")
    else:
        await message.answer(f"✅ <b>{name}</b> dan muhim belgisi olindi!")

@dp.message_handler(commands=["pin"])
async def cmd_pin(message: Message):
    if not is_auth(message.from_user.id): return
    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /pin fayl_nomi")
        return
    await toggle_pin(message, args[1], 1)

@dp.message_handler(commands=["unpin"])
async def cmd_unpin(message: Message):
    if not is_auth(message.from_user.id): return

```

```

    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /unpin fayl_nomi")
        return
    await toggle_pin(message, args[1], 0)

@dp.message_handler(commands=["pinned"])
async def cmd_pinned(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, "AND pinned=1")

# — NOMI O'ZGARTIRISH —————
@dp.message_handler(commands=["rename"])
async def cmd_rename(message: Message, state: FSMContext):
    if not is_auth(message.from_user.id): return
    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /rename eski_nom")
        return
    await state.update_data(old_name=args[1])
    await RenameState.waiting_new_name.set()
    await message.answer(f"✎ '<b>{args[1]}</b>' uchun yangi nom kiriting:")

@dp.message_handler(state=RenameState.waiting_new_name)
async def process_rename(message: Message, state: FSMContext):
    data = await state.get_data()
    old_name = data.get("old_name")
    new_name = message.text.strip()
    async with aioredislite.connect(DB_NAME) as db:
        await db.execute("UPDATE files SET file_name=? WHERE file_name=?", (new_n
        await db.commit()
    await state.finish()
    await message.answer(f"✅ <b>{old_name}</b> → <b>{new_name}</b> nomi o'zgarti

# — SARALASH —————
@dp.message_handler(commands=["bysize"])
async def cmd_bysize(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, order="size DESC")

@dp.message_handler(commands=["bytime"])
async def cmd_bytime(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, order="date DESC")

@dp.message_handler(commands=["byname"])
async def cmd_byname(message: Message):

```

```

        if not is_auth(message.from_user.id): return
        await show_files(message, order="file_name ASC")

# — KATEGORIYA —————
@dp.message_handler(commands=["videos"])
async def cmd_videos(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, "AND category='video'")

@dp.message_handler(commands=["photos"])
async def cmd_photos(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, "AND category='photo'")

@dp.message_handler(commands=["apps"])
async def cmd_apps(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, "AND category IN ('apk','ipa')")

@dp.message_handler(commands=["audios"])
async def cmd_audios(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, "AND category='audio'")

@dp.message_handler(commands=["docs"])
async def cmd_docs(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message, "AND category='doc'")

@dp.message_handler(commands=["list"])
async def cmd_list(message: Message):
    if not is_auth(message.from_user.id): return
    await show_files(message)

# — STATISTIKA —————
async def send_stats(message):
    async with aiosqlite.connect(DB_NAME) as db:
        async with db.execute("SELECT COUNT(*), SUM(size) FROM files") as cursor:
            total_count, total_size = await cursor.fetchone()
        async with db.execute("SELECT COUNT(*) FROM files WHERE category='video'") as cursor:
            video_count = (await cursor.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM files WHERE category='photo'") as cursor:
            photo_count = (await cursor.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM files WHERE category IN ('apk','ipa')") as cursor:
            app_count = (await cursor.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM files WHERE category='audio'") as cursor:
            audio_count = (await cursor.fetchone())[0]

```

```

        async with db.execute("SELECT COUNT(*) FROM files WHERE category='doc'")
            doc_count = (await cursor.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM files WHERE category='other'")
            other_count = (await cursor.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM files WHERE pinned=1") as cur
            pinned_count = (await cursor.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM folders") as cursor:
            folder_count = (await cursor.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM trash") as cursor:
            trash_count = (await cursor.fetchone())[0]

    await message.answer(
        f"📊 <b>Statistika</b>\n\n"
        f"📁 Jami fayllar: <b>{total_count or 0}</b>\n"
        f"📁 Umumiy hajm: <b>{format_size(total_size or 0)}</b>\n\n"
        f"🎬 Videolar: {video_count}\n"
        f"🖼️ Rasmlar: {photo_count}\n"
        f"🎵 Audio: {audio_count}\n"
        f"📄 Hujjatlar: {doc_count}\n"
        f"📱 APK/IPA: {app_count}\n"
        f"📁 Boshqalar: {other_count}\n\n"
        f"📌 Muhim fayllar: {pinned_count}\n"
        f"📁 Papkalar: {folder_count + 1}\n"
        f"🗑️ Savatda: {trash_count}"
    )

@dp.message_handler(commands=["stats"])
async def cmd_stats(message: Message):
    if not is_auth(message.from_user.id): return
    await send_stats(message)

# — YORDAM —————
async def send_help(message):
    await message.answer(
        "❓ <b>Yordam</b>\n\n"
        "📁 <b>Fayl yuborish:</b>\n"
        "Istalgan fayl, rasm, video, audio yuboring\n\n"
        "📁 <b>Ro'yxat:</b>\n"
        "/list — barcha fayllar\n"
        "/videos /photos /audios /docs /apps\n\n"
        "🔍 <b>Qidirish:</b>\n"
        "/search nom\n\n"
        "📁 <b>Papkalar:</b>\n"
        "/newfolder nom — yaratish\n"
        "/folders — ro'yxat\n"
        "/folder nom — ko'rish\n"
        "/moveto fayl|papka — ko'chirish\n"
    )

```

```

        "/delfolder nom - o'chirish\n\n"
        "📌 <b>Muhim:</b>\n"
        "/pin nom | /unpin nom | /pinned\n\n"
        "✎ <b>Boshqa:</b>\n"
        "/rename nom - nom o'zgartirish\n"
        "/delete nom - savatga\n"
        "/trash - savat\n"
        "/emptytrash - savatni tozalash\n"
        "/bysize /bytime /byname - saralash\n"
        "/stats - statistika"
    )

@dp.message_handler(commands=["help"])
async def cmd_help(message: Message):
    await send_help(message)

# — ADMIN —————
@dp.message_handler(commands=["broadcast"])
async def cmd_broadcast(message: Message):
    if message.from_user.id != ADMIN_ID:
        await message.answer("❌ Ruxsat yo'q!")
        return
    args = message.text.split(maxsplit=1)
    if len(args) < 2:
        await message.answer("Misol: /broadcast xabar matni")
        return
    text = args[1]
    count = 0
    for user_id in authorized_users:
        try:
            await bot.send_message(user_id, f"🔊 <b>Xabar:</b>\n\n{text}")
            count += 1
        except:
            pass
    await message.answer(f"✅ {count} ta foydalanuvchiga yuborildi!")

# — WEBHOOK —————
async def on_startup(dp):
    await create_db()
    await bot.set_webhook(WEBHOOK_URL)
    print(f"✅ Bot ishga tushdi! Webhook: {WEBHOOK_URL}")

async def on_shutdown(dp):
    await bot.delete_webhook()
    print("❌ Bot to'xtatildi!")

if __name__ == "__main__":

```

```
start_webhook(  
    dispatcher=dp,  
    webhook_path=WEBHOOK_PATH,  
    on_startup=on_startup,  
    on_shutdown=on_shutdown,  
    host=WEBAPP_HOST,  
    port=WEBAPP_PORT,  
)
```